

Trường Đại học Giao Thông Vận Tải thành phố Hồ Chí Minh
Viện Công nghệ thông tin và Điện, Điện tử

Chương 3.

Các phương pháp dựa trên giá trị

Tiến sĩ Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn



Học tăng cường chênh lệch tạm thời (Temporal difference RL)

Học tăng cường Chênh lệch Tạm thời (**Temporal Difference Learning - TD**) là một trong những ý tưởng cốt lõi và sáng tạo nhất của Reinforcement Learning, là sự kết hợp giữa phương pháp **Monte Carlo (MC)** và **Dynamic Programming (DP)**.

So sánh TD với Monte Carlo và Dynamic Programming

Tiêu chí	TD Learning	Monte Carlo	Dynamic Programming
Cần model môi trường	Không	Không	Có
Cập nhật	Sau mỗi bước	Sau mỗi tập	Dựa trên toàn bộ trạng thái
Tốc độ học	Nhanh	Chậm	Rất nhanh
Độ nhiễu	Trung bình	Cao	Thấp
Dùng bootstrap	Có	Không	Có

Temporal Difference Learning

Value Prediction

- TD(0)

- TD(λ)

- n-step TD

Control

- On-policy

 - SARSA

 - Expected SARSA

 - n-step SARSA

 - SARSA(λ)

- Off-policy

 - Q-learning

 - Double Q-learning

 - Watkins Q(λ)

 - DQN

Học tăng cường chênh lệch tạm thời (Temporal difference RL)

Học tăng cường Chênh lệch Tạm thời (**Temporal Difference Learning - TD**) là một trong những ý tưởng cốt lõi và sáng tạo nhất của Reinforcement Learning, là sự kết hợp giữa phương pháp **Monte Carlo (MC)** và **Dynamic Programming (DP)**.

So sánh TD với Monte Carlo và Dynamic Programming

Tiêu chí	TD Learning	Monte Carlo	Dynamic Programming
Cần model môi trường	Không	Không	Có
Cập nhật	Sau mỗi bước	Sau mỗi tập	Dựa trên toàn bộ trạng thái
Tốc độ học	Nhanh	Chậm	Rất nhanh
Độ nhiễu	Trung bình	Cao	Thấp
Dùng bootstrap	Có	Không	Có

Temporal Difference Learning

Value Prediction

- TD(0)

- TD(λ)

- n-step TD

Control

- On-policy

 - SARSA

 - Expected SARSA

 - n-step SARSA

 - SARSA(λ)

- Off-policy

 - Q-learning

 - Double Q-learning

 - Watkins Q(λ)

 - DQN

Temporal Difference Learning Concepts

Future States

Value influenced by future outcomes



Incomplete Sequences

Learning from partial outcomes



Bootstrapping

Updates based on prior estimates



Temporal Difference Learning

Core method for incremental learning



Học tăng cường chênh lệch tạm thời (Temporal difference RL)

Tiêu chí	TD(0)	n-step TD	TD(λ)
Số bước nhìn trước	1 bước	n bước	Vô hạn (giảm dần theo λ)
Tốc độ cập nhật	Nhanh nhất	Trung bình	Nhanh
Độ chính xác	Thấp hơn	Cao hơn TD(0)	Cao nhất
Độ nhiễu	Thấp	Trung bình	Thấp
Độ phức tạp	Đơn giản nhất	Trung bình	Phức tạp nhất
Bộ nhớ cần	Rất ít	Lưu n bước	Lưu toàn bộ traces
Khi $\lambda = 0$	= TD(0)	—	= TD(0)
Khi $\lambda \rightarrow 1$	—	—	$\approx$ Monte Carlo

Sử dụng các thuật toán:

TD(0) khi:

- Cần thuật toán đơn giản
- Bộ nhớ hạn chế
- Môi trường ổn định

n-step TD khi:

- Muốn cân bằng giữa TD và Monte Carlo
- Cần giảm độ chệch (bias)
- Chuỗi phần thưởng ngắn–trung bình

TD(λ) khi:

- Chuỗi trạng thái dài
- Phần thưởng xuất hiện trễ
- Cần hội tụ nhanh & ổn định
- Muốn hiệu suất tốt nhất

n-step Reinforcement Learning

n-step RL là nhóm phương pháp học tăng cường thuộc họ Temporal Difference, được thiết kế nhằm dung hòa giữa hai cách học truyền thống là cập nhật ngay sau mỗi bước như Temporal Difference Learning và chờ đến khi kết thúc toàn bộ tập như Monte Carlo.

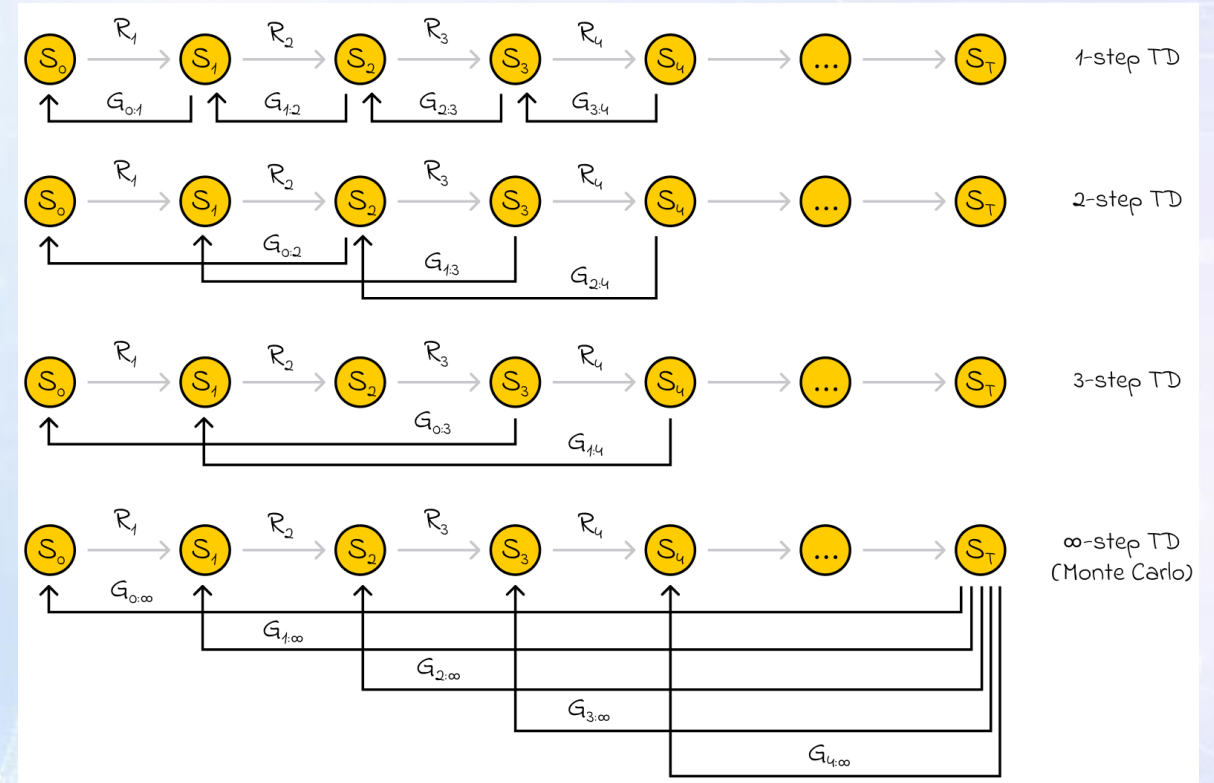
Ý tưởng cốt lõi của n-step là sử dụng một số hữu hạn n bước tương lai để đánh giá trạng thái hiện tại, nhờ đó thuật toán có thể nhìn xa hơn so với phương pháp một bước nhưng vẫn cập nhật sớm hơn so với cách chờ hết tập.

Khái niệm n-step return dùng để chỉ tổng phần thưởng tích lũy trong n bước tiếp theo kể từ thời điểm hiện tại, sau đó cộng thêm giá trị ước lượng của trạng thái tại bước thứ n .

Giá trị này được xem như mục tiêu để điều chỉnh ước lượng hiện tại.

- Nếu số bước quan sát càng nhiều thì thông tin thu được càng phản ánh chính xác phần thưởng thực tế, tuy nhiên thời gian cập nhật cũng chậm hơn.
- Ngược lại, nếu số bước nhỏ thì việc học diễn ra nhanh hơn nhưng mức độ chính xác của ước lượng có thể giảm.

Do đó n-step return tạo ra một cơ chế cân bằng giữa tốc độ học và độ tin cậy của thông tin.



Thuật toán n-step TD

Mục tiêu: Ước lượng hàm giá trị trạng thái $V(s)$ cho một chính sách đã cho.

Bước 1. Khởi tạo

Khởi tạo giá trị $V(s)$ cho mọi trạng thái s một cách tùy ý.

Chọn các tham số gồm tốc độ học α , hệ số chiết khấu γ và số bước nhìn

trước n .

Bước 2. Bắt đầu mỗi tập

Đặt trạng thái ban đầu S_0 .

Đặt thời điểm t bằng 0.

Khởi tạo danh sách lưu trạng thái và phần thưởng.

Bước 3. Tương tác với môi trường

Thực hiện hành động theo chính sách tại trạng thái hiện tại.

Quan sát phần thưởng nhận được và trạng thái kế tiếp.

Lưu trạng thái và phần thưởng vào chuỗi.

Tăng thời điểm t lên một đơn vị.

Bước 4. Kiểm tra điều kiện cập nhật

Khi số bước thu thập được lớn hơn hoặc bằng n , thực hiện cập nhật cho trạng thái tại thời điểm $t - n$.

Bước 5. Tính n-step return

Tính tổng phần thưởng của n bước kế tiếp có chiết khấu theo thời gian.

Cộng thêm giá trị ước lượng của trạng thái tại bước thứ n .

Giá trị này là mục tiêu để cập nhật.

Bước 6. Cập nhật giá trị trạng thái

Điều chỉnh giá trị trạng thái theo sai lệch giữa mục tiêu vừa tính và giá trị hiện tại.

Việc cập nhật được thực hiện theo tốc độ học α .

Bước 7. Trượt cửa sổ thời gian

Dịch mốc thời gian sang bước kế tiếp và tiếp tục quá trình tương tác với môi trường.

Bước 8. Kết thúc tập

Khi trạng thái kết thúc xuất hiện, tiếp tục cập nhật cho các trạng thái còn lại chưa đủ n bước bằng cách sử dụng phần thưởng còn lại trong tập.

Bước 9. Lặp lại

Quay lại Bước 2 cho đến khi đạt điều kiện dừng huấn luyện.

Ghi chú

Nếu n bằng một, thuật toán trở thành TD một bước.

Nếu n tăng đến chiều dài toàn bộ tập, phương pháp tiến gần đến Monte Carlo.

Giá trị n càng lớn thì thuật toán càng nhìn xa hơn nhưng cần nhiều bộ nhớ và thời gian tính toán hơn.

Pseudocode — n-step TD

n-step TD for estimating $V \approx v_\pi$

Input: a policy π

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

Initialize and store $S_0 \neq \text{terminal}$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \dots$:

| If $t < T$, then:

| Take an action according to $\pi(\cdot | S_t)$

| Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

| If S_{t+1} is terminal, then $T \leftarrow t + 1$

| $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

| If $\tau \geq 0$:

| $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

| If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$

($G_{\tau:\tau+n}$)

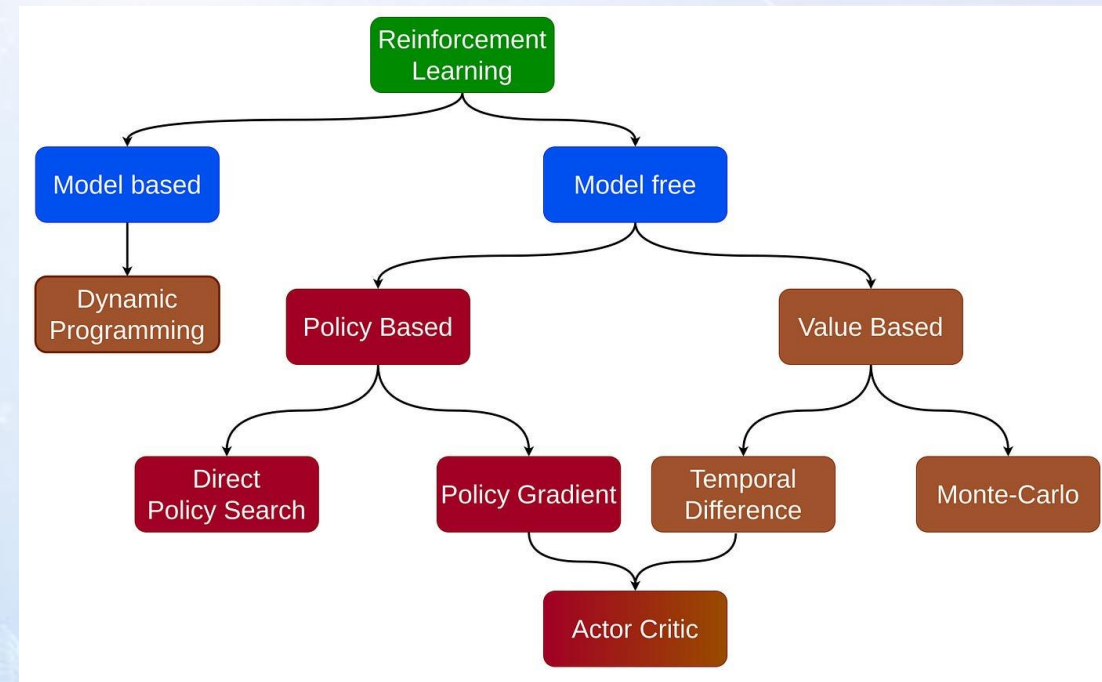
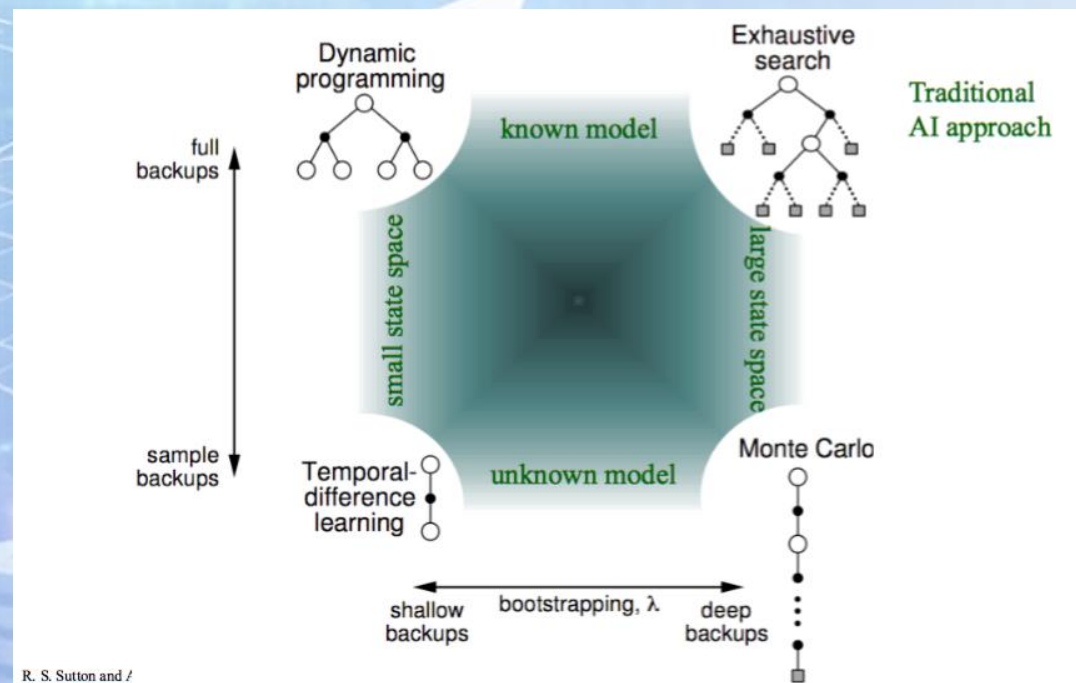
| $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$

Until $\tau = T - 1$

Quan hệ giữa Monte Carlo và TD Learning

n-step Learning thể hiện rõ mối quan hệ trung gian giữa Monte Carlo và Temporal Difference Learning.

- Khi n bằng một, phương pháp này trở thành Temporal Difference một bước vì chỉ sử dụng phần thưởng ngay lập tức và giá trị ước lượng của trạng thái kế tiếp.
- Khi n tăng dần và tiến đến độ dài toàn bộ tập, phương pháp sẽ dần giống Monte Carlo vì lúc này việc cập nhật dựa hoàn toàn trên chuỗi phần thưởng thực tế mà không cần dùng bootstrap.



Như vậy, n -step tạo thành một phổ liên tục nối liền hai trường phái học tăng cường quan trọng: học nhanh nhưng có độ chệch cao của Temporal Difference và học chính xác nhưng phương sai lớn của Monte Carlo

Ví dụ minh họa

Bài toán

Chuỗi trạng thái:

$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \text{Terminal}$

Reward nhận được sau mỗi bước:

Transition	Reward
$S_0 \rightarrow S_1$	0
$S_1 \rightarrow S_2$	0
$S_2 \rightarrow S_3$	0
$S_3 \rightarrow T$	+1

Thông số:

- $\gamma = 0.9$
- $\alpha = 0.1$
- $n = 2$ (2-step TD)

Giá trị khởi tạo:

$V(S_0)=0$

$V(S_1)=0$

$V(S_2)=0$

$V(S_3)=0$

Công thức 2-step return

$$G_t^{(2)} = r_{t+1} + \gamma r_{t+2} + \gamma^2 V(S_{t+2})$$

Cập nhật:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t^{(2)} - V(S_t)]$$

Các bước cập nhật

Bước $t = 0$ (cập nhật S_0)

$$G_0^{(2)} = r_1 + \gamma r_2 + \gamma^2 V(S_2) = 0 + 0 + 0.9^2 \times 0 = 0$$

$$V(S_0) = 0 + 0.1(0 - 0) = 0$$

Bước $t = 1$ (cập nhật S_1)

$$G_1^{(2)} = r_2 + \gamma r_3 + \gamma^2 V(S_3) = 0 + 0 + 0.9^2 \times 0 = 0$$

$$V(S_1) = 0 + 0.1(0 - 0) = 0$$

Bước $t = 2$ (cập nhật S_2)

$$G_2^{(2)} = r_3 + \gamma r_4 + \gamma^2 V(\text{Terminal})$$

$$\text{Terminal} \Rightarrow V = 0$$

$$G_2^{(2)} = 0 + 0.9 \times 1 + 0 = 0.9$$

$$V(S_2) = 0 + 0.1(0.9 - 0) = \mathbf{0.09}$$

Bước $t = 3$ (không đủ 2 bước phía sau)

Khi gần cuối episode, n-step trở thành Monte Carlo rút gọn:

$$G_3 = r_4 = 1$$

$$V(S_3) = 0 + 0.1(1 - 0) = \mathbf{0.1}$$

t mạnh hơn

Ví dụ minh họa

Bảng tóm tắt cập nhật

t	State	n-step Return G_t	V(s) cũ	V(s) mới
0	S0	0	0	0
1	S1	0	0	0
2	S2	0.9	0	0.09
3	S3	1	0	0.1

Giá trị sau 1 episode

$$V(S0) = 0$$

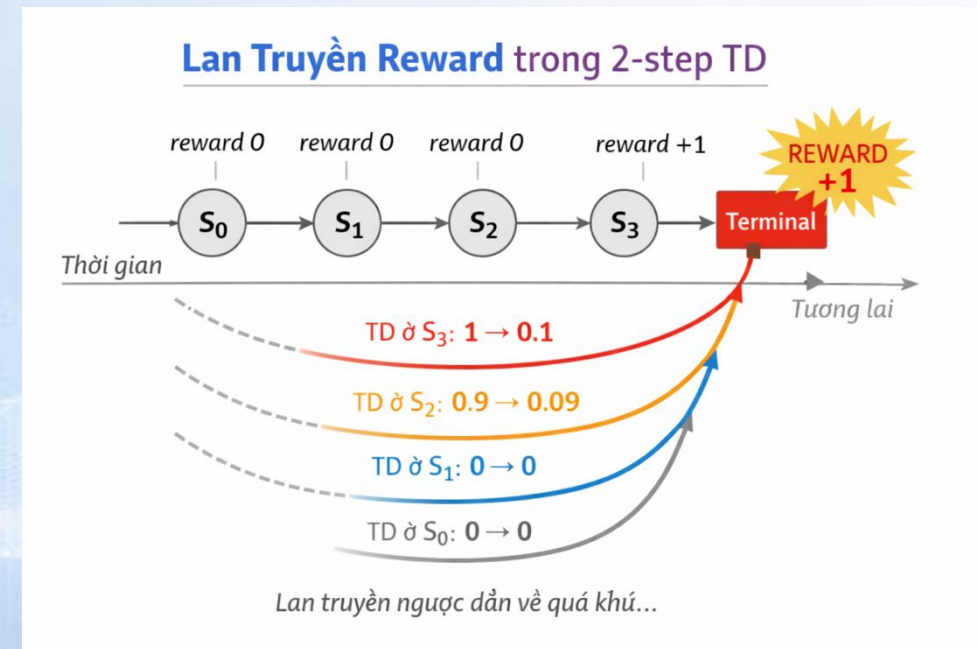
$$V(S1) = 0$$

$$V(S2) = 0.09$$

$$V(S3) = 0.1$$

Reward +1 ở cuối **lan truyền ngược dần**:
Terminal $\rightarrow S3 \rightarrow S2 \rightarrow S1 \rightarrow S0$

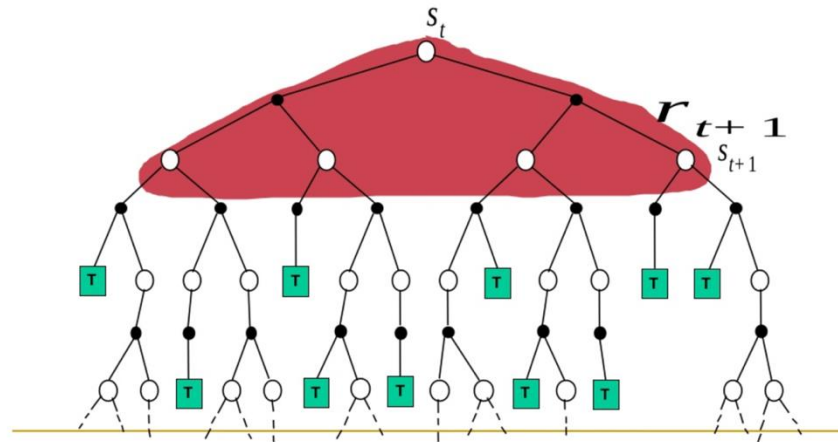
- α nhỏ, $n = 2$ (nhìn ngắn) $\rightarrow$ reward lan truyền chậm.
- n lớn hơn $\rightarrow$ lan nhanh hơn
- α lớn hơn $\rightarrow$ cập nhật nhanh hơn



Minh hoạ học giá trị trạng thái

Dynamic Programming

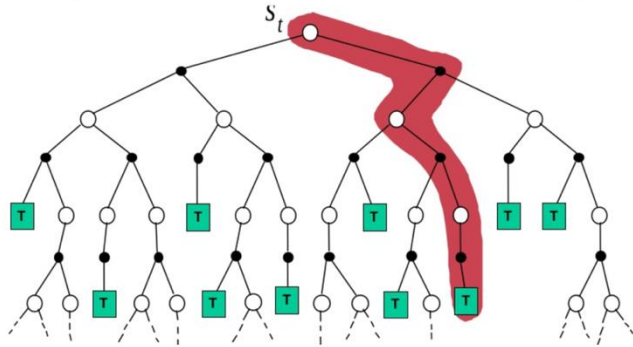
$$V(s_t) \leftarrow E_{\pi} \{ r_{t+1} + \gamma V(s_{t+1}) \}$$



Monte Carlo Learning

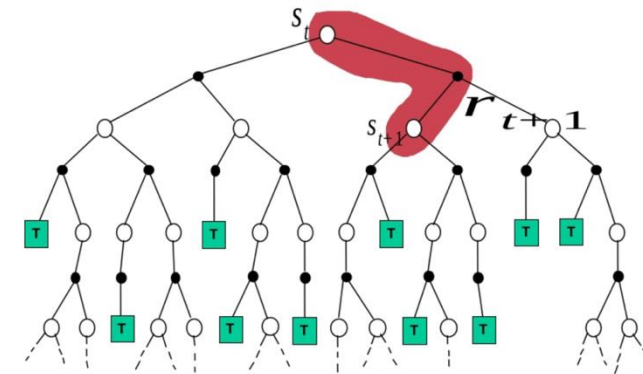
$$V(s_t) \leftarrow V(s_t) + \alpha [R_t - V(s_t)]$$

where R_t is the actual return following state s_t .



Temporal Difference Learning

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$



Hàm Q xấp xỉ (Q-function approximation)

1. Hàm Q trong Reinforcement Learning

Trong Reinforcement Learning, hàm giá trị hành động:

$Q(s,a)$ = mức độ “tốt” khi thực hiện hành động a tại trạng thái s và tiếp tục theo chính sách.

2. Hạn chế của bảng Q (Q-table)

Bùng nổ số lượng trạng thái

Nếu:

- 1.000.000 trạng thái

- 100 hành động

→ cần lưu **100 triệu giá trị Q**

Không xử lý được trạng thái liên tục

Ví dụ:

- Robot: tọa độ $(x,y) \in \mathbb{R}$

- Xe tự lái: vận tốc, góc lái liên tục

→ Không thể lập bảng.

Không có khả năng tổng quát hóa

Hai trạng thái gần giống nhau vẫn được xem là hoàn toàn khác nhau.

3. Ý tưởng xấp xỉ hàm Q

Thay bảng tra cứu bằng **mô hình tham số**:

$$Q(s,a) \approx Q(s,a; \theta)$$

Trong đó:

- θ : tham số mô hình

- RL trở thành bài toán học tham số

→ Giảm bộ nhớ

→ Tổng quát hóa tốt hơn

→ Áp dụng cho không gian lớn

Linear Function Approximation

Linear Function Approximation biểu diễn đặc trưng bằng vector $\phi(s,a)$:

$$Q(s,a;\theta) = \theta^T \phi(s,a)$$

Ưu điểm

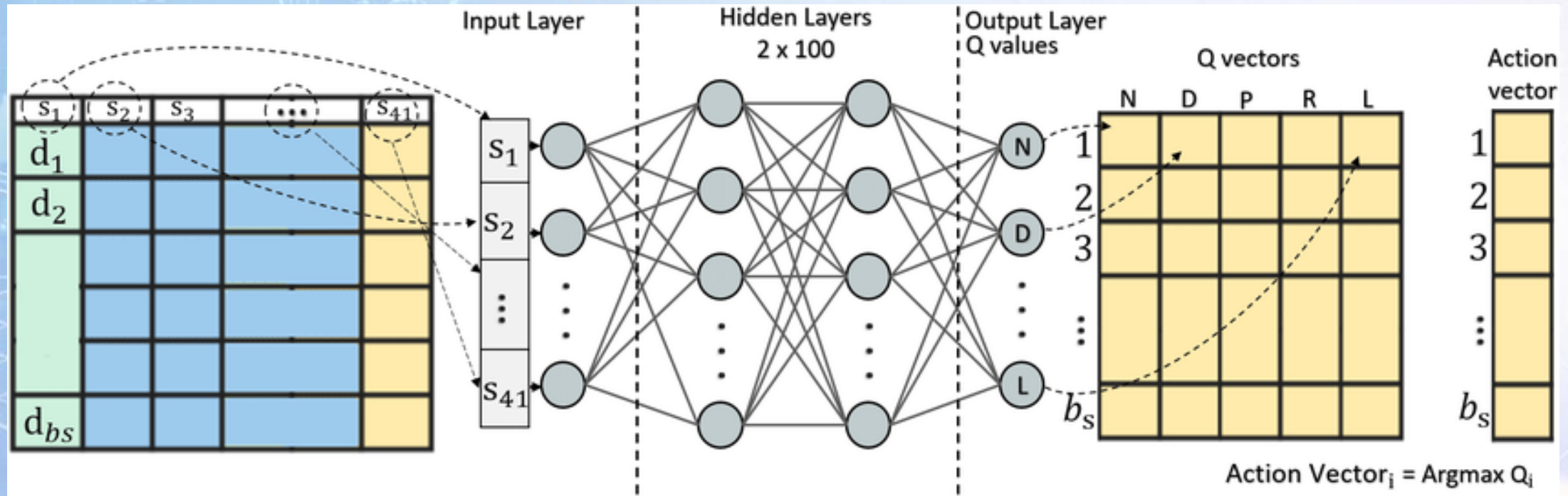
- Đơn giản
- Dễ huấn luyện
- Hội tụ ổn định

Nhược điểm

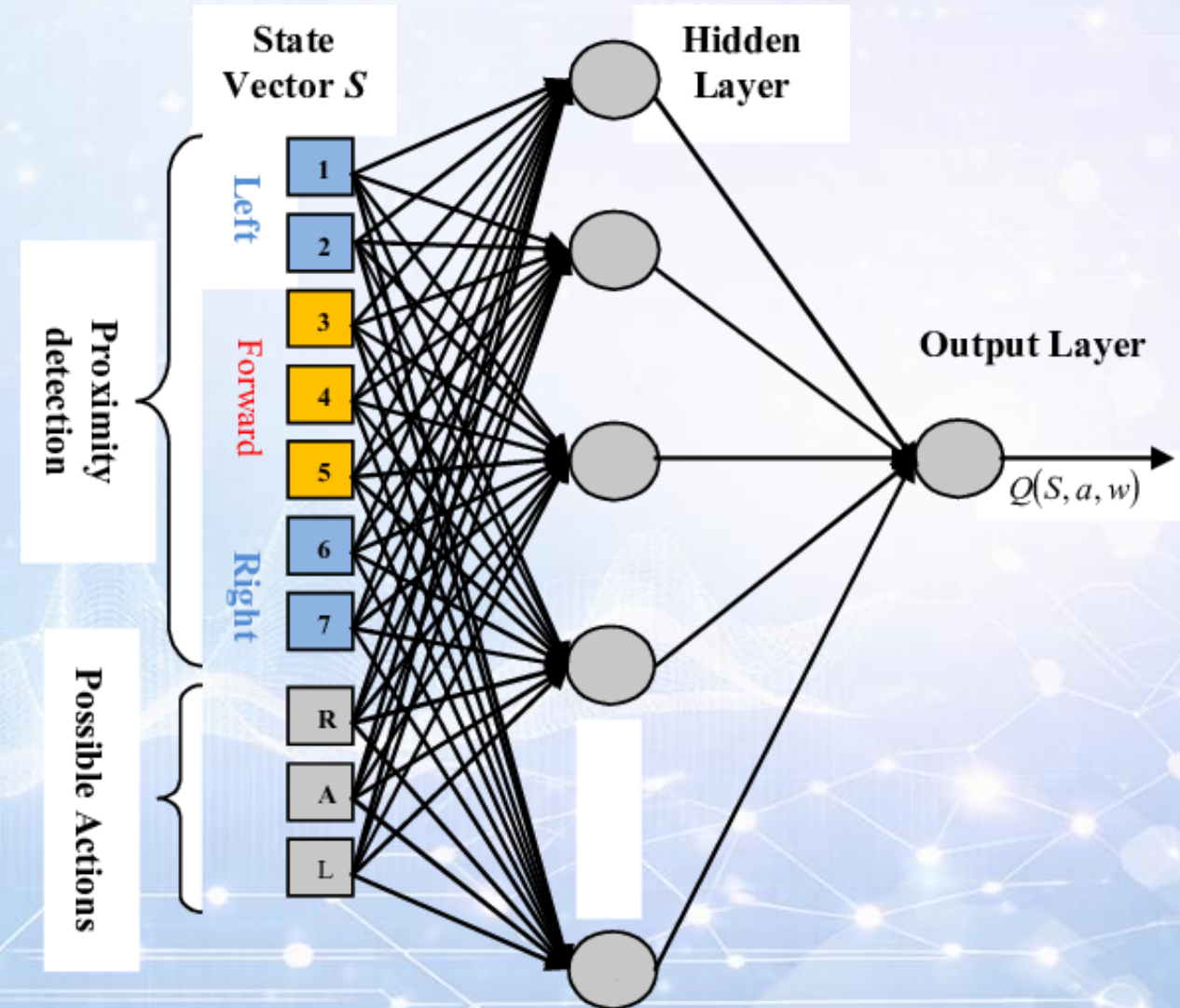
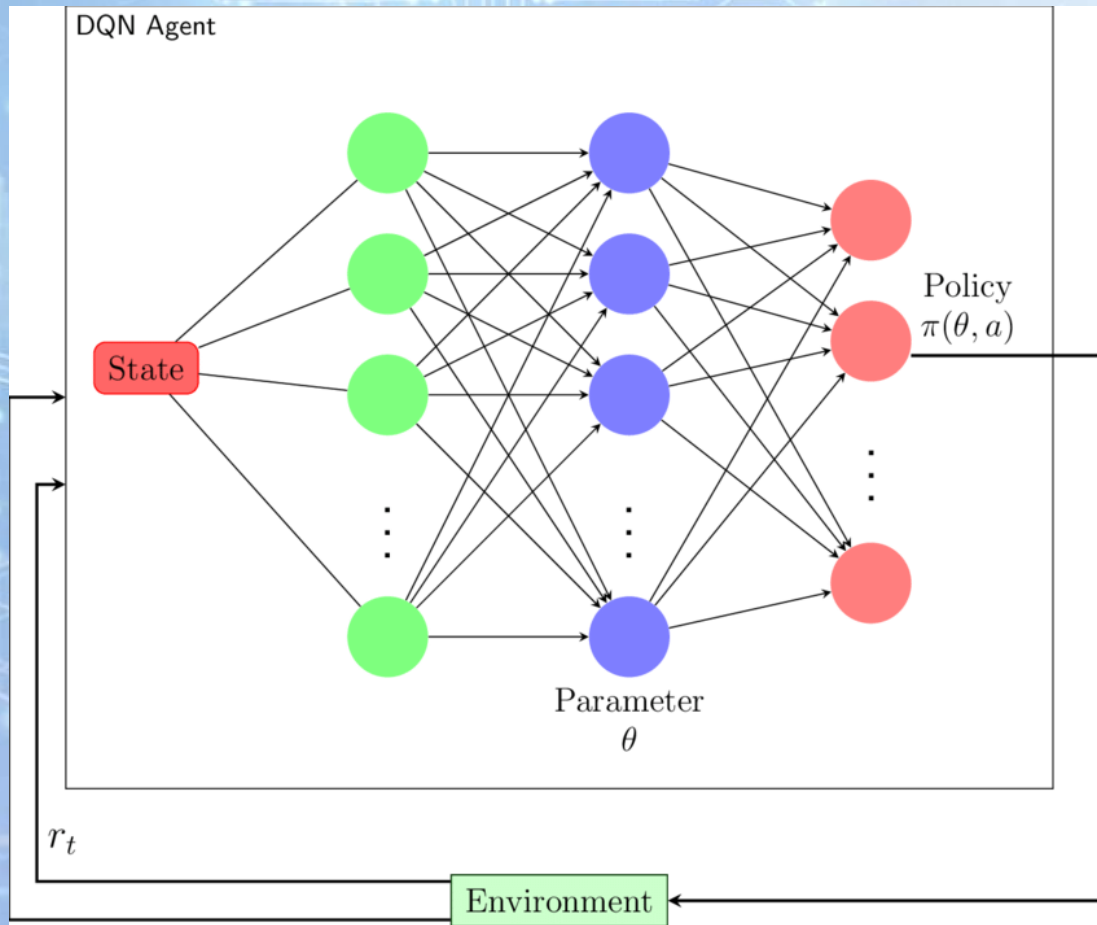
- Không biểu diễn quan hệ phi tuyến
 - Khả năng mô hình hóa hạn chế
- Phù hợp bài toán nhỏ, đặc trưng rõ ràng

Phương pháp	Biểu diễn	Tổng quát hóa	Phù hợp
Q-table	Bảng tra cứu	Không	Không gian nhỏ
Linear Approx	Tuyến tính	Trung bình	Bài toán đơn giản
Neural Network	Phi tuyến	Tốt	Không gian lớn
DQN	Deep Learning + Q-learning	Rất tốt	Bài toán phức tạp

Hàm Q xấp xỉ (Q-function approximation)



Minh hoạ Q-network



Định hình phần thưởng (Reward shaping)

Reward Shaping là kỹ thuật thiết kế hoặc bổ sung phần thưởng phụ vào hàm reward gốc của môi trường nhằm hướng dẫn agent học nhanh hơn và hiệu quả hơn.

Trong nhiều bài toán RL:

- Reward rất **thưa (sparse)** → agent khó học
- Reward chỉ xuất hiện ở cuối → học rất chậm
- Không có tín hiệu hướng dẫn → agent thử sai nhiều

Mục đích áp dụng Reward Shaping:

- Tăng tốc hội tụ
- Giảm số lần trial-and-error
- Cải thiện hiệu quả học trong môi trường phức tạp

Ý tưởng:

Thay vì dùng reward gốc:

$$R(s' a' s')$$

Ta thêm một thành phần shaping:

$$R'(s' a' s') = R(s' a' s') + F(s' a' s')$$

Trong đó:

F : reward bổ sung (shaping function)

Ví dụ: Bài toán tìm đường trong mê cung.

Reward gốc: đến đích: +100, các bước khác: 0 → Agent rất khó học vì không biết được đi đúng hay sai cho đến khi kết thúc

Sau khi áp dụng reward shaping:

- Đi gần đích hơn: +1
- Đi xa đích: -1
- Đến đích: +100

→ Agent có tín hiệu liên tục → học nhanh hơn

5. Ưu điểm của Reward Shaping

- Tăng tốc học
- Dễ hội tụ hơn
- Tăng stability trong training

6. Nhược điểm

- Thiết kế sai → agent học lệch mục tiêu
- Phụ thuộc domain knowledge
- **Reward Hacking:** Nếu hàm định hình không được thiết kế kỹ, agent có thể tìm cách "lách luật" để nhận nhiều phần thưởng phụ mà không thực hiện mục tiêu chính.

Định hình phần thưởng (Reward shaping)

Các loại Reward Shaping phổ biến:

1. Heuristic Shaping. Dựa trên kiến thức con người:

- Khoảng cách đến mục tiêu
- Hướng di chuyển
- Các quy tắc domain

2. Potential-based Reward Shaping: áp dụng hàm shaping có dạng:

$$F(s', s) = \gamma \Phi(s') - \Phi(s),$$

trong đó:

$\Phi(s)$:hàm thế năng (potential function)

Trong các phương pháp **Temporal Difference**, việc học diễn ra thông qua **TD error** – sai lệch giữa giá trị dự đoán và mục tiêu. Reward Shaping tác động trực tiếp bằng cách **thay đổi tín hiệu reward**, từ đó **làm thay đổi TD error và toàn bộ quá trình cập nhật**.

Có nghĩa là:

- Reward Shaping không chỉ “thêm reward” mà thực chất là:
- Thay đổi trực tiếp TD error → thay đổi toàn bộ dynamics học của TD Learning

Ảnh hưởng với Q-learning

TD error phụ thuộc vào max Q

Reward shaping làm thay đổi giá trị Q ngay từ đầu

→ ảnh hưởng trực tiếp đến policy học được

Với SARSA

TD error phụ thuộc action thực tế

→ Reward shaping ảnh hưởng cả hành vi exploration

Những rủi ro nếu thiết kế sai

Vì Reward Shaping đi thẳng vào TD update nên:

Có thể làm lệch hướng học

Agent tối ưu reward phụ thay vì mục tiêu chính

Gây bias mạnh trong value function